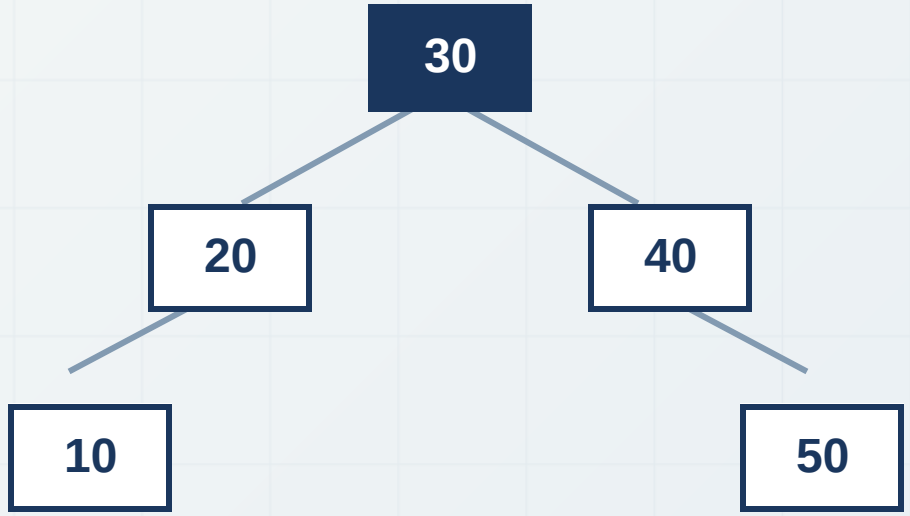
APPLIED DATA STRUCTURES

Case Study of Optimal Binary Search Tree Using Dynamic Programming

How search frequencies guide the structure of an efficient BST.

Presented by: Deokar Aditya Sandip
Class / Division: TY-A
College: Sanjivani College of Engineering, Kopargaon



A SIMPLE BINARY SEARCH TREE

Why Search Structure Matters

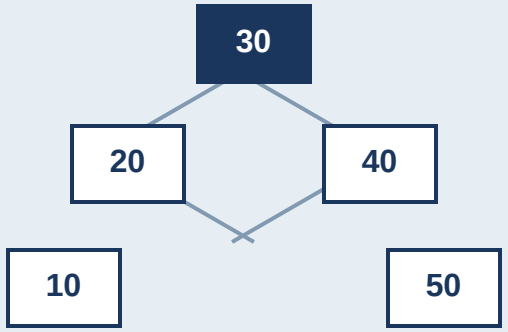
Searching is one of the most common operations in computer science.

A **Binary Search Tree** stores ordered data, but its performance depends on its shape.

When frequently searched keys sit close to the root, the average search path becomes shorter.

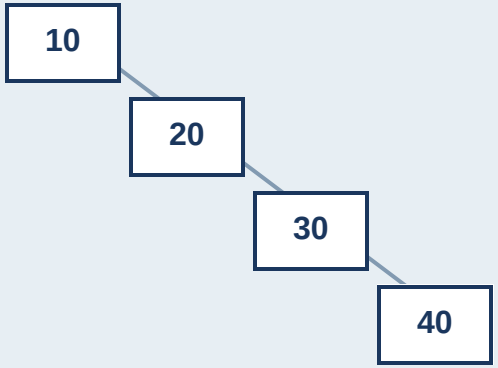
This motivates the Optimal Binary Search Tree.

Balanced BST



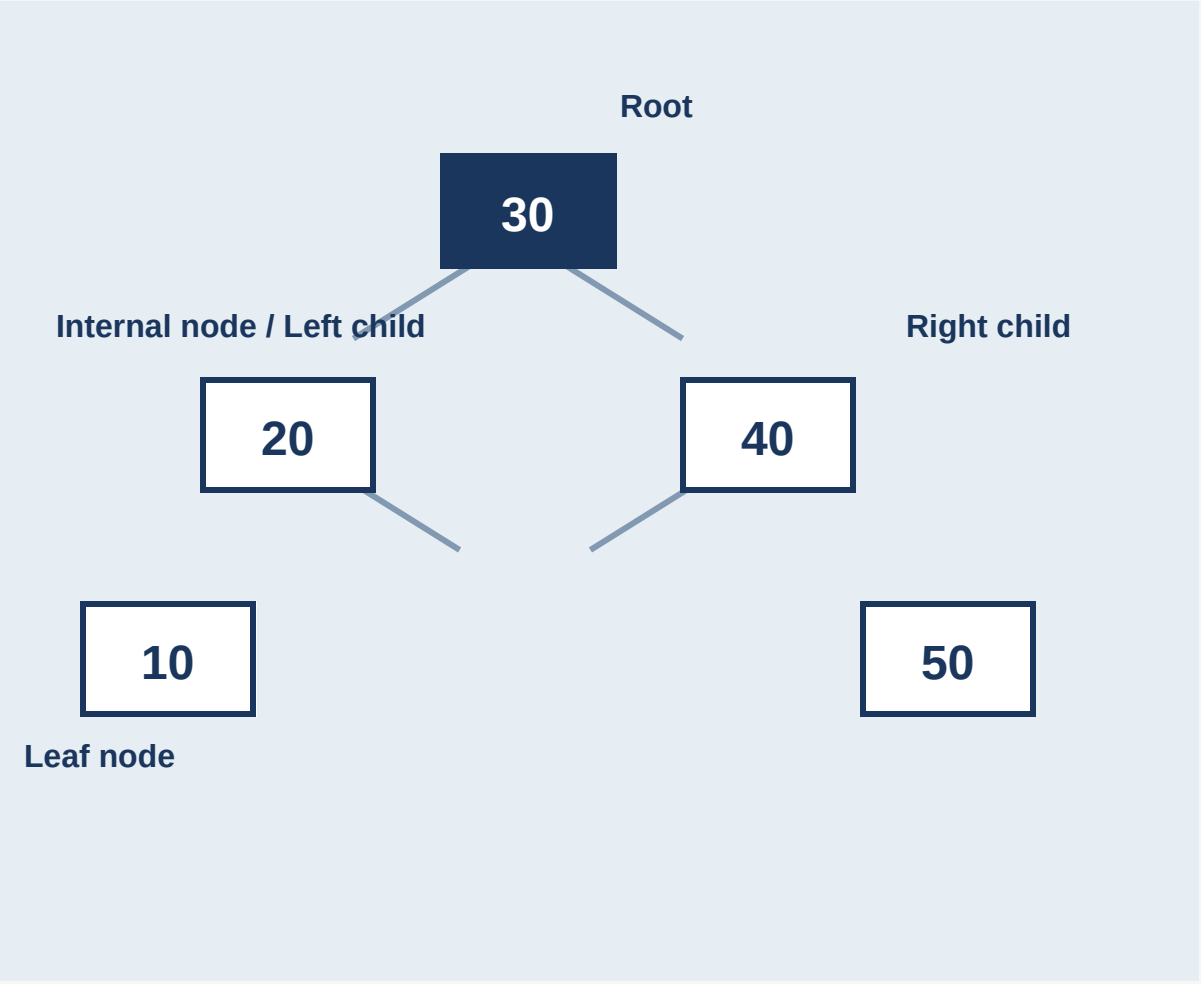
Shorter average paths

Unbalanced BST



Longer search paths

Binary Trees and Binary Search Trees



A **binary tree** is hierarchical: each node has at most two children.

In a **BST**, left-subtree keys are smaller and right-subtree keys are greater than the root.

Both subtrees must themselves satisfy the BST rule.

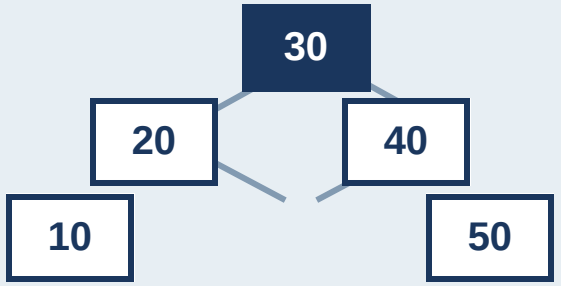
Example: keys 10, 20, 30, 40, 50.
Searching for 50 follows **30** → **40** → **50**.

The Ordinary BST Problem

The insertion order can produce very different tree shapes—even when the keys are identical.

Balanced BST

Keys are distributed across levels.



Short paths **Lower average cost**

Skewed BST

Sorted insertion creates a chain.



Long paths **Higher average cost**

Balanced tree
shorter search paths.

Skewed tree
more comparisons per search.

Key insight
an ordinary BST is not automatically optimal.

What Makes a BST Optimal?

An Optimal Binary Search Tree **is arranged so that the expected search cost is minimum, considering key frequencies or probabilities.**

01 Frequently searched keys should be closer to the root.

02 Rarely searched keys can be placed deeper in the tree.

03 The objective is to minimize the weighted path length.

Minimum expected search cost = Optimal BST

Search frequency determines how valuable a shallow position is.

Frequency-weighted placement

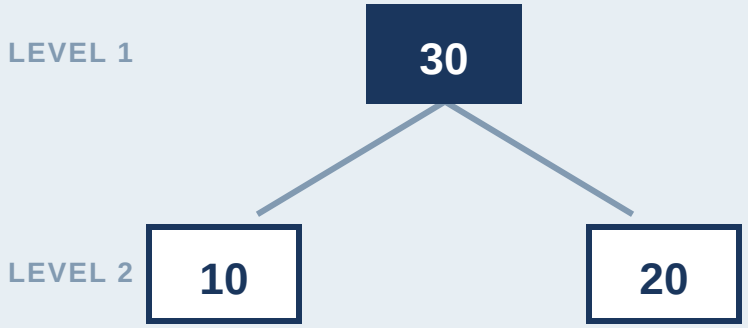
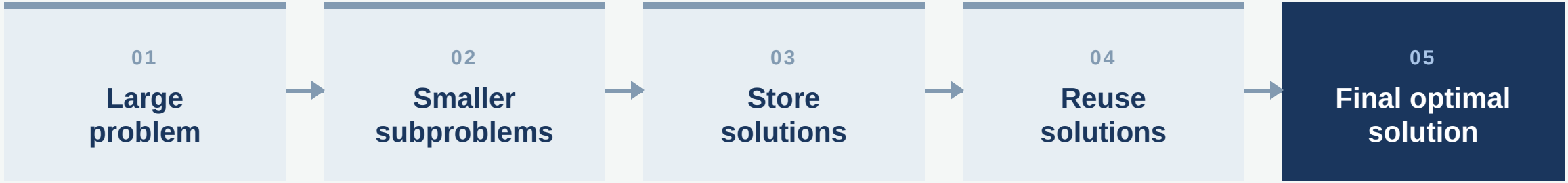


Illustration: **if key 30 is searched most often, placing it near the root can lower total cost.**

Applications Dictionaries • Symbol tables • Compiler structures • Static lookup systems • Database-indexing concepts

Dynamic Programming: The Core Idea

Dynamic Programming (DP) solves a complex problem by breaking it into smaller, overlapping subproblems and storing their solutions for reuse.



Overlapping subproblems

The same smaller ranges or states appear repeatedly. DP solves each one once and records the result.

Avoid repeated computation

Optimal substructure

An optimal solution can be built from optimal solutions to its smaller subproblems.

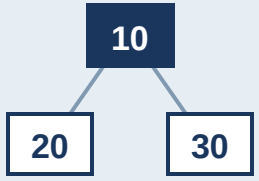
Build the best answer from the best parts

Why OBST Uses Dynamic Programming

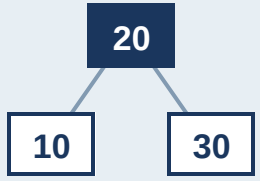
Finding the best root means comparing every valid root for each key range.

For a range such as **[10, 20, 30]**, each candidate root creates a different left and right subtree.

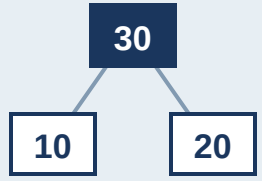
Root = 10



Root = 20



Root = 30



1 **Choose a candidate root**

Try each key as the root of the current range.

2 **Evaluate both subtrees**

Calculate the best cost for the left and right ranges.

3 **Add the weighted contribution**

Include the frequency contribution of the current range.

4 **Keep the minimum**

Store the best result so it can be reused by larger ranges.

OBST → overlapping subproblems + optimal substructure → Dynamic Programming

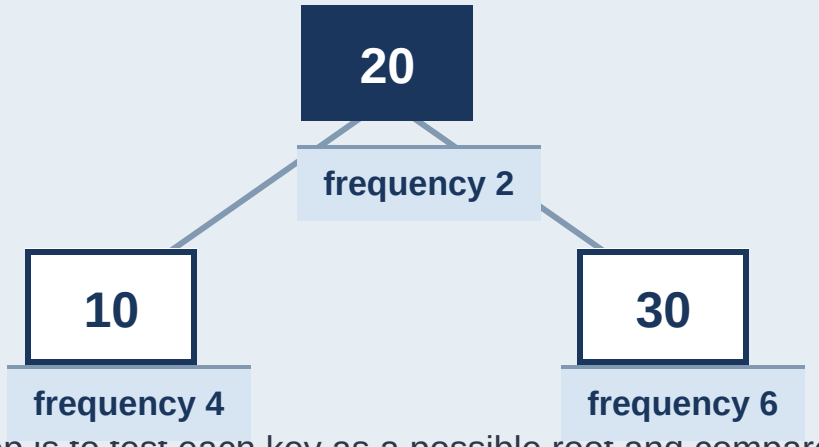
Case Study: Keys and Frequencies

We begin with three ordered keys and their successful-search frequencies.

Key	Frequency
10	4
20	2
30	**6**

Observation: key 30 is searched most often, so placing it near the root may reduce the overall cost.

Ordered keys with search frequencies



The next step is to test each key as a possible root and compare the weighted search costs.

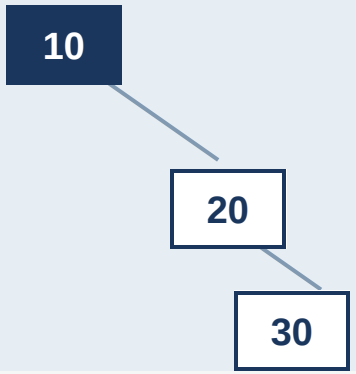
Cost = Σ (frequency × level) | root level = 1

Comparing the Three Root Choices

Using **Cost = Σ (frequency × level)**, with the root at level 1.

Case 1: Root = 10

HIGHER COST

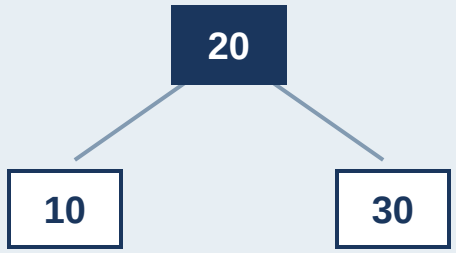


10 at level 1 → 4×1
20 at level 2 → 2×2
30 at level 3 → 6×3

Total = 26

Case 2: Root = 20

OPTIMAL

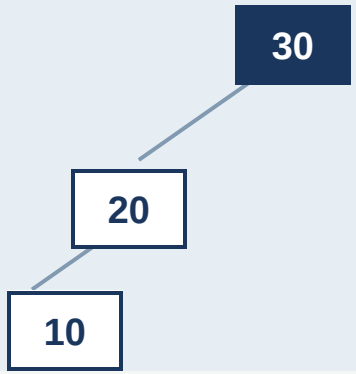


20 at level 1 → 2×1
10 at level 2 → 4×2
30 at level 2 → 6×2

Total = 22

Case 3: Root = 30

OPTIMAL



30 at level 1 → 6×1
20 at level 2 → 2×2
10 at level 3 → 4×3

Total = 22

Minimum search cost

22

Important result

Two structures are equally optimal for this frequency set.

Decision rule

Choose the root arrangement with the lowest weighted path length.

OBST Dynamic-Programming Table

C[i][j] stores the minimum search cost for the ordered keys from position *i* through position *j*.

Cost table for keys 10, 20, 30

C[i][j]	10	20	30
10	4	8	22
20		2	10
30			6

Reading the table: diagonal cells are single-key base cases; cells above the diagonal represent larger key ranges.

How larger ranges are built

1. Initialize single-key trees: **4, 2, 6**.
2. For each two-key range, try both possible roots and store the smaller cost: **8** and **10**.
3. For the full range **[10, 20, 30]**, try roots 10, 20, and 30.

Full-range comparison

Root 10 → 26 | Root 20 → 22 | Root 30 → 22

Minimum C[1][3] = 22

The table records the best cost; a root table can record which key produced it.

Algorithm and Working Flow

OBST algorithm

1 Sort the keys if necessary.
BST order must be preserved.

2 Store the search frequencies.
Use successful-search frequencies $p_1 \ldots p_n$.

3 Initialize the DP table.
Single-key ranges are the base cases.

4 Process ranges of increasing length.
Build larger solutions from smaller ones.

5 Try every key as a possible root.
Evaluate the left and right subtrees.

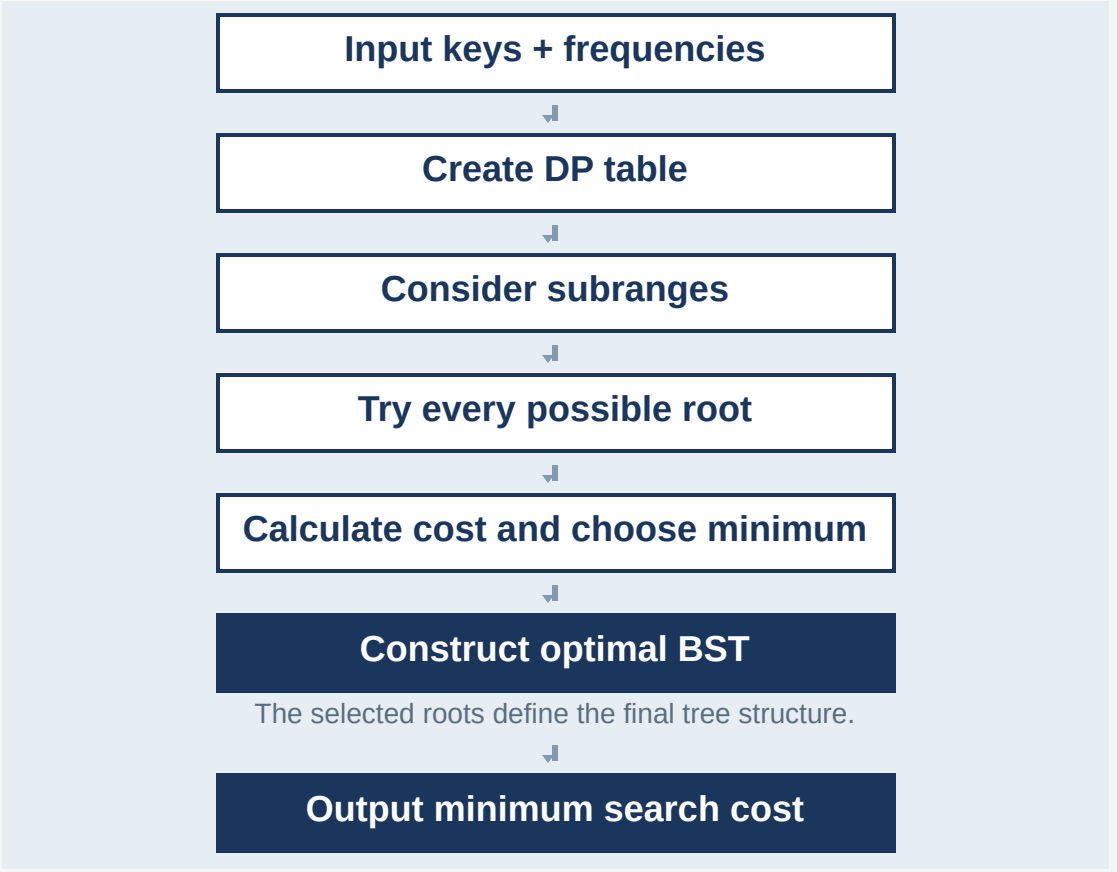
6 Calculate and compare costs.
Left cost + right cost + frequency contribution.

7 Store the minimum and selected root.
Root information supports reconstruction.

8 Reconstruct the optimal BST.
Output the minimum expected search cost.

The table records both the best cost and the root that achieved it.

Working flow



Benefits, Limitations, and Conclusion

Advantages

01 Minimizes expected search cost using known frequencies.

02 Places frequently accessed keys closer to the root.

03 Systematically compares all valid root arrangements.

04 Avoids repeated calculations through dynamic programming.

05 Works well for static or relatively stable datasets.

Limitations

01 Requires reliable frequency or probability information.

02 More complex than constructing an ordinary BST.

03 Dynamic programming requires additional memory.

04 The tree may need rebuilding when frequencies change significantly.

05 Less suitable when search patterns change continuously.

What the case study demonstrates

BST structure matters

Different shapes create different search costs even for the same keys.

Frequencies guide placement

The most frequently searched keys benefit from shallow levels.

DP finds the minimum

Comparing possible roots reveals the best arrangement and cost.

> “The goal is not just to build a BST, but to build the BST that makes the most frequently used searches as efficient as possible.”

OPTIMAL BINARY SEARCH TREE — FINAL TAKEAWAY